

Imobiliare.ro Scraper — How It Works

Overview

This is a **browser console scraper** — it runs directly in Chrome/Firefox DevTools on the imobiliare.ro listings page. It does two things:

1. Scrapes the **listing cards** on the search results page
 2. Visits **each individual listing URL** to grab deeper details
 3. Downloads everything as a **JSON file**
-

Step 1 — XPath Helpers

```
javascript
```

```
function getXPathNode(xpath, context = document) { ... }  
function getXPathNodes(xpath, context = document) { ... }
```

Two utility functions that wrap the browser's native `document.evaluate()` API:

- `getXPathNode` — returns the **first** matching element
- `getXPathNodes` — returns **all** matching elements as a snapshot

XPath is used instead of `querySelector` in some places because it supports traversal like `ancestor::` (going UP the DOM tree), which CSS selectors cannot do.

Step 2 — Finding All Listing Cards

```
javascript
```

```
const listings = getXPathNodes('//a[starts-with(@id,"listing-link-")]');
```

This finds every anchor tag whose `id` starts with `"listing-link-"` — these are the clickable links on each property card. From each link, we walk UP the DOM to find the parent card container:

```
javascript
```

```
const card = getXPathNode('./ancestor::div[contains(@class,"relative")][1]', link);
```

This grabs the nearest ancestor `<div>` with class `relative`, which wraps all the card's data.

Step 3 — Card-Level Fields

These are scraped directly from the search results page using `data-cy` attributes (reliable, site-maintained selectors):

Field	Selector
Title	<code>h3 span.text-title</code>
Price	<code>[data-cy="card-price"]</code>
Bedrooms	<code>[data-cy="card-bedroom_count"]</code>
Usable Surface	<code>[data-cy="card-usable_surface"]</code>
Floor Number	<code>[data-cy="card-floor_number"]</code>
Building Type	<code>[data-cy="card-building_type"]</code>
URL	<code>link.href</code>

`data-cy` attributes are added by developers for testing purposes — they're more stable than class names which can change with UI updates.

Step 4 — Cross-Page Fetch (Detail Page)

```
javascript

async function fetchListingDetails(url) {
  const res = await fetch(url);
  const html = await res.text();
  const doc = new DOMParser().parseFromString(html, 'text/html');
  ...
}
```

For each listing URL, the scraper:

1. **Fetches** the full HTML of the detail page using `fetch()`
2. **Parses** it into a DOM document using `DOMParser`
3. **Queries** specific elements from that parsed document

This is done without opening any new tabs — everything happens in memory.

Fields extracted from the detail page:

Field	Selector	Notes
Address	<code>[data-cy="listing-address"]</code>	City / county
Heading	<code>[data-cy="listing-heading"]</code>	Full listing title
Partitioning	<code>[data-cy="listing-amenities-excerpt-component"]</code>	e.g. Semidecomandat
Basic Details	<code>[data-cy="basic-info-section"]</code>	All detail rows joined
Amenities	<code>[data-cy="listing-amenities-component"]</code>	Utilities, features
Description	<code>[data-cy="listing-description-section"]</code>	First 200 characters

Step 5 — Rate Limiting Delay

```
javascript
await new Promise(r => setTimeout(r, 500));
```

After each listing fetch, the script waits **500 milliseconds** before moving to the next one. This prevents the site from detecting and blocking the scraper due to too many rapid requests. If you get blocked or errors, increase this to `1000` (1 second).

Step 6 — JSON Download

```
javascript
```

```
function downloadJSON(data) {  
    const blob = new Blob([JSON.stringify(data, null, 2)], { type: 'application/json' });  
    const a = document.createElement('a');  
    a.href = URL.createObjectURL(blob);  
    a.download = 'imobiliare_listings.json';  
    a.click();  
}
```

Once all listings are scraped, the data array is:

1. Converted to a formatted JSON string with `JSON.stringify(data, null, 2)`
2. Wrapped in a `Blob` (binary file object)
3. Attached to a hidden `<a>` tag and programmatically clicked to trigger a browser download

The file saves as `imobiliare_listings.json` in your downloads folder.

Output Structure

Each listing in the JSON looks like this:

```
json  
  
{  
  "index": 1,  
  "url": "https://www.imobiliare.ro/oferta/...",  
  "title": "Apartament 3 camere zona Fabric",  
  "heading": "Apartament 3 Camere | 85 mp | Etaj 2",  
  "address": "Timișoara, Județul Timiș",  
  "price": "89.900 €",  
  "bedrooms": "3 camere",  
  "buildingType": "Bloc de apartamente",  
  "floor": "Etaj 2 / 4",  
  "usableSurface": "85 mp",  
  "partitioning": "Semidecomandat",  
  "basicDetails": "Nr. cam.: 3 | Suprafață: 85 mp | ...",  
  "amenities": "Gaz, Curent, Apă, Canalizare, ...",  
  "description": "Apartament spatios situat in zona centrala..."  
}
```

How to Run

1. Open imobiliare.ro/vanzare-imobiliare/judetul-timis/timisoara in Chrome
 2. Press `F12` to open DevTools
 3. Click the **Console** tab
 4. Paste the full script and press `Enter`
 5. Wait for it to finish — you'll see progress logs like `⌚ Fetching listing 1 / 30...`
 6. The file `imobiliare_listings.json` will automatically download when done
-

Potential Issues

Issue	Cause	Fix
Fields showing <code>N/A</code>	Site updated their <code>data-cy</code> attributes	Re-run the debug snippet to find new values
Script gets blocked	Too many requests too fast	Increase delay from <code>500</code> to <code>1000</code> + ms
Download doesn't trigger	Browser blocked the popup	Allow popups for the site
CORS error on fetch	Site blocks cross-origin requests	Run the script on the imobiliare.ro page itself, not from another domain